

accDashboard Delivery Disposition Guide

accDashboard — Delivery Disposition Dashboard

Documentation, access guide, and deployment next steps

Repository	github.com/gwepsilon/accDashboard
Feature branch	<code>cursor-cloud/delivery-disposition-dashboard-351a</code>
Pull request	PR #2 — Delivery Disposition Dashboard
Document version	1.0
Date	28 September 2026

1. Executive summary

This delivery implements an **ACC-style Delivery Disposition Dashboard** for the accDashboard repository. The application aggregates recipient disposition counts from PostgreSQL and presents them as a **pastel status-square matrix** keyed by **delivery label** and **internal name**.

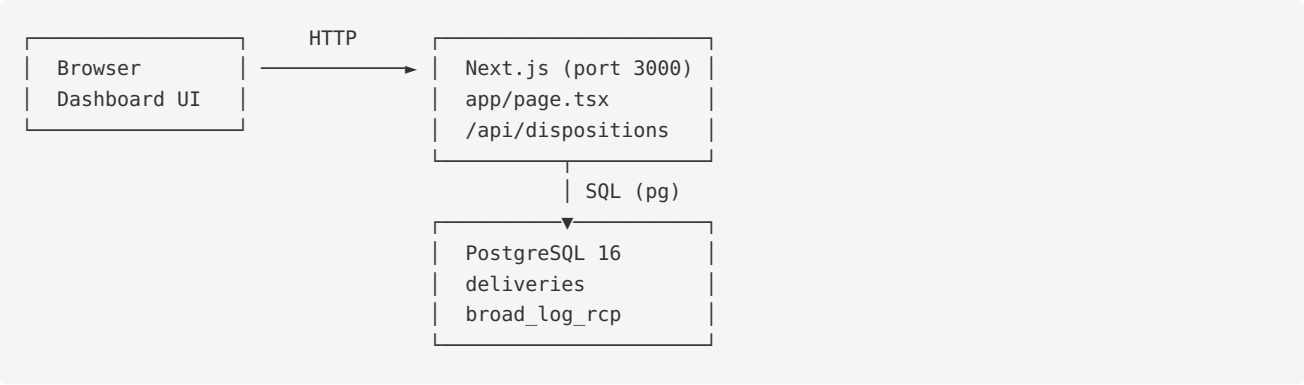
Capabilities delivered:

- Next.js 15 web application (App Router)
- PostgreSQL schema: `deliveries` and `broad_log_rcp` (modeled after ACC `broadLogRcp`)
- Seed data for five sample deliveries
- REST API: `GET /api/dispositions`
- Docker Compose for local PostgreSQL
- Cloud Agent environment scripts for install and database startup

Disposition statuses (fixed set):

Status	Typical meaning (operational)
Ignored	Recipient excluded or not targeted
Sent	Message handed off to channel
Failed	Delivery or rendering failure
Prepared	Staged / queued for send
Transmitted	Confirmed handoff to provider

2. Architecture overview



Key paths in the repository:

Path	Purpose
app/page.tsx	Dashboard UI (fetches API, renders matrix)
app/api/dispositions/route.ts	API route handler
lib/dispositions.ts	Aggregation logic
lib/db.ts	PostgreSQL connection pool
docker-compose.yml	Local database service
docker/init/01-schema.sql	Schema and enum type
docker/init/02-seed.sql	Demo seed data
.cursor/environment.json	Cloud Agent install/start/dev

3. Data model

Table: deliveries

Stores campaign or delivery metadata visible in the dashboard.

Column	Type	Description
id	SERIAL	Primary key
label	VARCHAR(255)	Human-readable delivery label (matrix row title)
internal_name	VARCHAR(255)	Unique internal identifier (ACC-style technical name)
created_at	TIMESTAMPTZ	Creation timestamp

Table: broad_log_rcp

Per-recipient disposition rows (ACC **broadLogRcp** pattern).

Column	Type	Description
id	BIGSERIAL	Primary key
delivery_id	INTEGER	FK → deliveries.id
recipient_key	VARCHAR(255)	Recipient identifier (e.g. rcp-1001)
status	disposition_status	One of the five statuses
event_at	TIMESTAMPZ	Last event time

Enum: disposition_status = Ignored , Sent , Failed , Prepared , Transmitted

4. How to access the application

4.1 Prerequisites

- **Node.js** 20 or newer
- **npm** (lockfile: package-lock.json)
- **Docker** (recommended) for PostgreSQL, *or* a local PostgreSQL 16 instance

4.2 Clone and checkout

```
git clone https://github.com/gwepsilon/accDashboard.git
cd accDashboard
git checkout cusor-cloud/delivery-disposition-dashboard-351a
# Or use main after PR #2 is merged
```

4.3 Install dependencies

```
npm ci
```

4.4 Start the database

Option A — Docker Compose (recommended):

```
npm run db:up
```

On first start, scripts in docker/init/ create tables and load seed data.

Option B — Local PostgreSQL:

Set DATABASE_URL (see .env.example) and apply:

```
psql "$DATABASE_URL" -f docker/init/01-schema.sql
psql "$DATABASE_URL" -f docker/init/02-seed.sql
```

Cloud Agents can run bash scripts/cloud-agent-start.sh , which uses Docker when available or bootstraps local PostgreSQL.

4.5 Run the application

Development:

```
npm run dev
```

Production build locally:

```
npm run build
npm run start
```

4.6 URLs and endpoints

Resource	URL (local default)
Dashboard UI	http://localhost:3000
Dispositions API	http://localhost:3000/api/dispositions

Default database connection (matches Docker Compose):

```
postgresql://acc:acc@localhost:5432/acc_dashboard
```

Override with environment variable:

```
DATABASE_URL=postgresql://USER:PASSWORD@HOST:5432/DATABASE
```

4.7 API reference: GET /api/dispositions

Response: 200 OK with JSON body.

```
{
  "statuses": ["Ignored", "Sent", "Failed", "Prepared", "Transmitted"],
  "rows": [
    {
      "deliveryId": 1,
      "label": "Spring Promo Blast",
      "internalName": "spring_promo_blast_2026",
      "counts": {
        "Ignored": 1,
        "Sent": 3,
        "Failed": 2,
        "Prepared": 1,
        "Transmitted": 1
      },
      "total": 8
    }
  ],
  "generatedAt": "2026-09-28T16:42:19.000Z"
}
```

Errors: 500 with { "error": "...", "detail": "..." } if the database is unreachable.

4.8 npm scripts

Script	Command	Purpose
dev	next dev -H 0.0.0.0 -p 3000	Development server
build	next build	Production build
start	next start -H 0.0.0.0 -p 3000	Run production server
lint	eslint .	Static analysis
db:up	docker compose up -d db	Start PostgreSQL container
db:down	docker compose down	Stop containers

5. Cloud Agents (Cursor)

If you use Cursor Cloud Agents on this repository:

1. **Install:** `bash scripts/cloud-agent-install.sh ( npm ci )`
2. **Start:** `bash scripts/cloud-agent-start.sh` (PostgreSQL)
3. **Dev terminal:** `npm run dev` (port **3000**)

Configuration: `.cursor/environment.json` .

6. Verification checklist

Use this before sharing or deploying:

- ☐ `npm run db:up` — database healthy (`pg_isready`)
- ☐ `curl -s http://localhost:3000/api/dispositions` returns JSON with five statuses
- ☐ Browser shows matrix with five delivery rows and pastel status cells
- ☐ `npm run build` succeeds
- ☐ `npm run lint` succeeds

7. Next steps to deploy for external access

The current repository is optimized for **local development** and **Cloud Agents**. To make the dashboard **reachable on the internet** (or your corporate network), complete the steps below.

7.1 Merge and stabilize the codebase

1. Review and merge [PR #2](#) into `main` .
2. Tag a release (e.g. `v0.1.0`) for traceable deployments.
3. Replace demo credentials in Docker Compose with secrets managed outside git.

7.2 Production database

Step	Action
1	Provision managed PostgreSQL (AWS RDS, Azure Database, Google Cloud SQL, Neon, Supabase, etc.).
2	Run <code>docker/init/01-schema.sql</code> against the production database (or add a migration tool: Flyway, Prisma Migrate, Drizzle).
3	Do not run <code>02-seed.sql</code> in production unless you want demo data; instead ingest real <code>deliveries</code> and <code>broad_log_rcp</code> from your ACC or ETL pipeline.
4	Store <code>DATABASE_URL</code> in a secrets manager (AWS Secrets Manager, Vault, GitHub Actions secrets).
5	Enable TLS to the database, restrict network access (VPC / private link), and use least-privilege DB users (read-only for reporting if applicable).

7.3 Containerize the Next.js application

Today only the **database** is in Docker Compose. For production you typically add an **app** service:

1. Add a `Dockerfile` with multi-stage build: `npm ci` → `npm run build` → `run` `npm run start` (or standalone Next.js output).
2. Extend `docker-compose.yml` with an `app` service linking to `db` on an internal network.
3. Pass `DATABASE_URL` via environment variables (never commit secrets).

Alternative: deploy to **Vercel** or similar and attach a hosted Postgres (Neon/Supabase); set `DATABASE_URL` in the platform dashboard.

7.4 Hosting options (summary)

Pattern	Best for	Notes
Vercel + managed Postgres	Fastest path for Next.js	Native Next support; DB separate
Docker on VM / ECS / App Service	Full control, private network	Run app + optional sidecar DB
Kubernetes	Enterprise scale	Ingress + secrets + HPA
Internal only	Compliance	VPN or private ingress, no public DNS

7.5 Networking and DNS

1. Register a hostname (e.g. `acc-dashboard.yourcompany.com`).
2. Terminate **TLS** at load balancer or ingress (ACM, Let’s Encrypt).
3. Expose port **443** → app on **3000** (or run app on 8080 behind reverse proxy).
4. If the API is public, consider rate limiting and authentication (see security).

7.6 CI/CD pipeline

Suggested pipeline stages:

1. **Lint & build** — `npm run lint` , `npm run build` on every PR.
2. **Integration test** — spin up Postgres service container, apply schema, hit `/api/dispositions` .
3. **Deploy** — push container image or trigger Vercel deploy on merge to `main` .
4. **Smoke test** — HTTP 200 on `/` and `/api/dispositions` post-deploy.

GitHub Actions example services: `postgres:16-alpine` with init scripts mounted from `docker/init/` .

7.7 Security and compliance

Area	Recommendation
Authentication	Add SSO (OIDC/SAML) or API gateway auth before exposing externally; dashboard currently has no auth .
Secrets	Rotate <code>acc</code> / <code>acc demo</code> passwords; use strong credentials and IAM/database auth where supported.
Data	<code>broad_log_rcp</code> may contain PII — classify data, encrypt at rest, audit access.
Headers	Add security headers via Next.js middleware or reverse proxy.
CORS	Restrict if API is consumed only by same-origin UI.

7.8 Observability

- **Health endpoint:** add `/api/health` checking DB connectivity (recommended for load balancers).
- **Logging:** structured logs from Next.js; ship to CloudWatch, Datadog, or ELK.
- **Metrics:** request latency, DB pool usage, error rate on `/api/dispositions` .

- **Alerts:** on 5xx rate or DB connection failures.

7.9 Connecting to real ACC data

Longer-term integration (not in current MVP):

1. ETL or streaming job from ACC **broadLogRcp** exports into `broad_log_rcp`.
2. Sync `deliveries` from campaign metadata APIs or warehouse tables.
3. Schedule refresh (batch nightly or near-real-time) and show `generatedAt` on the dashboard (already present).

7.10 Suggested deployment sequence

Merge PR → Provision Postgres → Apply schema → Set DATABASE_URL
→ Build container or Vercel project → Deploy to staging
→ Smoke test → Add TLS + DNS → Add auth → Production cutover

8. Support and references

Item	Link or location
README (quick start)	<code>README.md</code> in repository root
Environment example	<code>.env.example</code>
GitHub repository	https://github.com/gwepsilon/accDashboard
Pull request	https://github.com/gwepsilon/accDashboard/pull/2

End of document